Table of Contents

CommitmentOS Build Plan 5

All Things Agentic Hackathon — Implementation-Ready Competition Plan

Version: 5.0
Plan date: August 10, 2026
Working product name: CommitmentOS
Primary track: The Taskmaster
Secondary award target: Best Architectural Design
Primary persona: A person managing deadline-driven project work through Gmail and Google Calendar
P0 evidence sources: Gmail and Google Calendar only
Pinned model: Gemini 3.5 Flash (`gemini-3.5-flash`)
Agent framework: Google Agent Development Kit 2.x, Python graph workflow
Primary runtime: One Python/FastAPI Cloud Run service serving the authenticated API and compiled React dashboard
Durable application state: Firestore-owned domain state and event records
Execution rule: Every cloud invocation is bounded; human approval and external-action results continue through new durable observations
External action identity rule: Every work block receives one stable Calendar event ID derived from its immutable work-block identity; plan revisions never change that external identity
Execution-control rule: Durable system controls can pause observation and hold every not-yet-started Calendar mutation; each executor rechecks those controls immediately before external I/O
P0 completion rule: Explicit user confirmation is authoritative; sent-email completion inference is deferred until the P0 gate passes

Version 5 Revision Summary

Version 5 preserves the complete Version 4 P0 scope and implementation schedule. It adds six competition-hardening decisions:

- An explicit public Calendar-webhook boundary for the single Cloud Run service, separated from OIDC-authenticated Pub/Sub and Cloud Tasks routes
- A formal distinction between authoritative facts and replaceable, provenance-carrying projections

- Stable Calendar event identity derived from immutable work-block identity rather than action or plan revisions
- A Phase 0 decision spike between unverified personal-use In-production OAuth and the seven-day External/Testing fallback
- A multi-message competition fixture that makes Gemini's ownership, deadline, and commitment-identity reasoning visible
- Durable monitoring and automatic-action controls that hold queued mutations and force revalidation after resume

1. Executive Decision

Build CommitmentOS as an evidence-backed, capacity-aware commitment controller—not as an inbox summarizer, chatbot, or generic personal assistant.

The complete P0 product is one closed loop:

> Detect a commitment in Gmail → preserve its evidence and ownership → confirm effort → reserve Calendar capacity → observe a conflict → reconcile and minimally repair the plan → explain the action → verify completion.

The Reconciliation Engine is the center of the product. Gemini 3.5 Flash interprets ambiguous human language and produces structured proposals. Deterministic services retain authority over identity resolution, state transitions, portfolio capacity, progress, risk, scheduling constraints, action policy, idempotency, and Calendar mutations.

The competition positioning is: **CommitmentOS is not an assistant that plans once. It is a controller that keeps a promise feasible as reality changes, until completion is verified.** The product must make Gemini's semantic role visible through evidence-backed ownership, deadline, and thread-update interpretation while keeping every consequential action under deterministic policy.

Every reconciliation run is a bounded cloud invocation. When user input is required, the run writes an approval or confirmation record and terminates safely. When an external Calendar action is approved, the run writes an outbox record, enqueues a Cloud Task, and terminates. The separate action executor performs the mutation, transactionally records the result and a new immutable observation, and enqueues follow-up reconciliation. Dashboard responses and action results therefore start fresh runs from durable Firestore state; P0 never depends on an in-memory Cloud Run process surviving a wait.

One durable execution-control document governs observation and action execution. Pausing automatic actions holds pending outbox work before Calendar I/O without

discarding intent or audit history. Resuming actions causes held records to be revalidated against current revisions before dispatch; stale intent is superseded rather than executed.

P0 does not require Canvas, dependency graphs, external follow-ups, Drive, multimodal input, or a general chat interface. Those features begin only after the Gmail-to-Calendar loop passes the reliability gate in Section 16.

2. Product Thesis and Competitive Boundary

2.1 Core promise

Turn scattered promises into continuously managed execution plans.

2.2 Positioning

Google Calendar shows when events happen. Gmail and Gemini can summarize information and surface action items. General agents such as Gemini Spark can execute user-directed Workspace workflows. CommitmentOS manages the longitudinal control state between a promise and its verified completion: provenance, ownership, remaining effort, shared capacity, feasibility, repair, and proof.

Its differentiation comes from the combination of:

- A durable commitment ledger rather than a transient summary
- Source-linked evidence and explicit ownership
- Confirmed effort and remaining-work state
- Portfolio-aware scheduling that prevents multiple commitments from claiming the same free time
- Persistent observation of real-world changes
- Minimal-change automatic plan repair
- Separate feasibility and dependency state
- Policy-controlled actions with undo
- Completion evidence instead of assuming elapsed time means success
- A decision timeline explaining every mutation

2.3 Product boundary

CommitmentOS will not spend P0 time on:

- Generic Gmail summaries
- A chatbot as the primary interface

- Daily briefs without actions
- Simple “create a Calendar event” behavior
- Meeting-time suggestions
- Content generation unrelated to commitment completion
- Broad Workspace automation
- Autonomous external messaging

2.4 P0 user scenario

The initial user is someone who makes deadline-bound promises over email and must fit the work around an already busy Calendar. P0 supports multiple active commitments for that one controlled user; portfolio planning must ensure shared free time is allocated only once. The same foundation can later serve students through Canvas and teams through shared dependencies, but those extensions do not define the initial product.

3. Product Outcomes

CommitmentOS should answer six questions for every active commitment:

1. What outcome was promised?
2. Who owns it and who benefits from it?
3. What source evidence supports the inference?
4. How much work remains before the deadline?
5. Is the commitment still achievable under the current Calendar?
6. What evidence shows that it was completed?

The product succeeds only when it manages all six questions as durable state.

For competition evidence, the dashboard should expose a small outcome strip derived from audit data:

- Active commitments kept feasible
- Work minutes reserved before deadlines
- Conflicts detected and repaired automatically
- Unaffected Calendar blocks preserved during repair
- Duplicate external actions prevented during replay tests
- Manual rescheduling actions avoided

The demo must pair those aggregate counters with one concrete outcome sentence, for example: “A new meeting endangered a three-hour commitment; CommitmentOS moved one block, preserved two approved blocks, and restored feasibility without

manual replanning.” These are measured proof-of-operation outcomes, not speculative productivity claims.

4. Commitment Lifecycle

4.1 Lifecycle states

```
candidate
    ↓
awaiting_confirmation
    ↓
active
    ↓
in_progress
    ↓
completion_candidate
    ↓
completed
```

Side states are `paused`, `dismissed`, and `canceled`. These lifecycle states are separate from risk and blocking state.

4.2 Feasibility risk

```
unknown | on_track | at_risk | critical | overdue
```

4.3 Blocking state

```
clear | waiting | blocked
```

Separating these dimensions prevents loss of information. A later P1 commitment can be both `blocked` by another person and `critical` because insufficient time remains.

4.4 Work-block execution state

```
planned → active → awaiting_check_in → completed
                  └──────────────────→ missed
planned ──────────────────────────────→ canceled
```

Calendar time passing is not proof that work happened. A block contributes to `verified_completed_minutes` only after explicit user confirmation. An elapsed unconfirmed block becomes `awaiting_check_in`, and a skipped block becomes `missed`; both trigger reconciliation without silently reducing remaining effort.

4.5 Completion invariant

`completed` is an explicit terminal business state supported by a stored `completion_evidence_id` and `completed_at`. While a commitment is active, remaining work is derived from confirmed effort minus verified minutes. Once completion is confirmed, the commitment remains closed even if the recorded verified minutes are lower than the original estimate; the system must not invent work minutes or reopen the commitment during a later reconciliation. Reopening requires an explicit user action that creates a new revision.

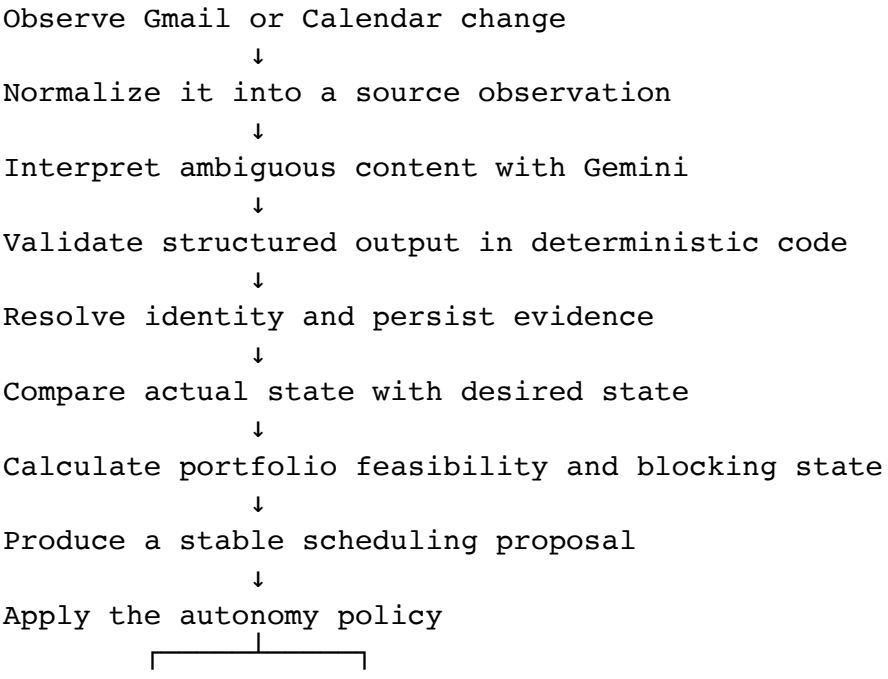
4.6 Desired versus actual state

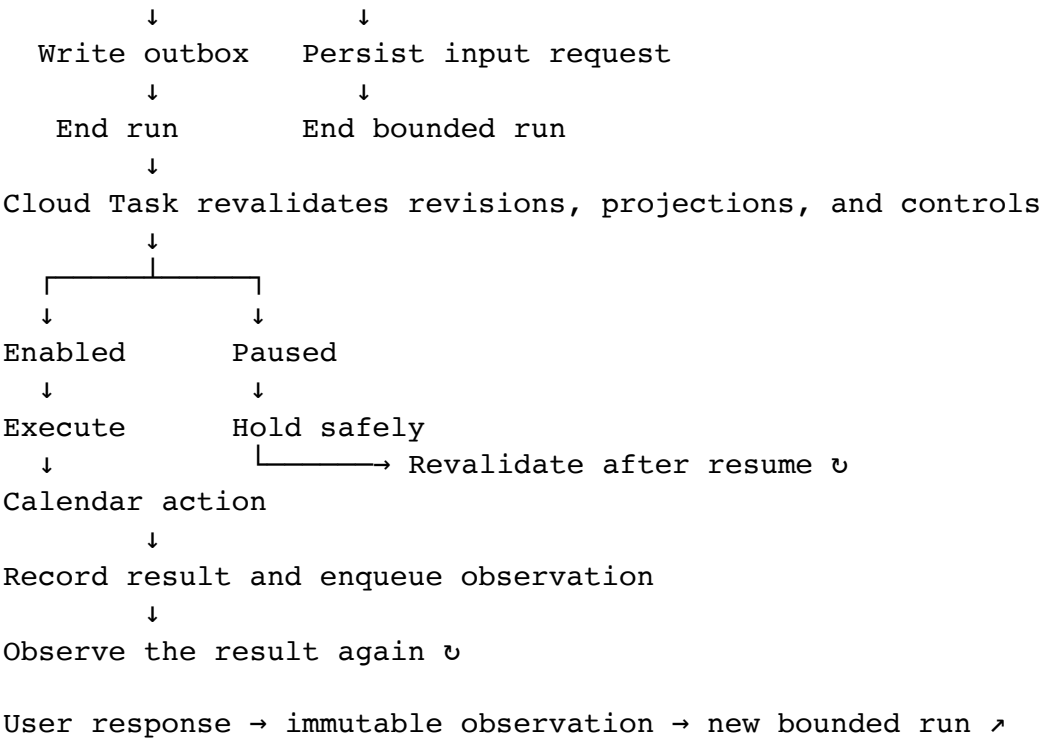
For each commitment, the Reconciliation Engine maintains:

- **Desired state:** confirmed deadline, remaining effort, portfolio allocation, policy, and valid work blocks required before the deadline
- **Actual state:** current Gmail evidence, Calendar events, verified work progress, work-block status, approvals, completion evidence, and synchronization state

Reconciliation compares the two. It acts only when a meaningful difference exists and records the exact before-and-after state.

5. Primary Autonomous Loop





This is an event-driven workflow, not an unbounded LLM loop or a long-lived web request. Every ADK run has an input observation, a bounded set of nodes, deterministic termination, and a durable outcome. Calendar mutations happen only in a separate replay-safe Cloud Task executor. Firestore is the source of truth between runs; ADK expresses reconciliation control flow but is never the durability boundary.

6. Locked P0 Scope

6.1 P0 capabilities

Capability	P0 acceptance condition
Gmail ingestion	New and sent-message changes reach the backend and can recover from missed notifications
Structured interpretation	Gemini returns schema-valid commitment proposals with source evidence
Ownership	The system distinguishes my commitment, request to me, commitment to me, and ambiguous language
Commitment identity	Thread updates modify or supersede the correct commitment without silent duplication
Effort confirmation	The user confirms or edits the proposed effort before the first Calendar plan
Calendar capacity	Busy time, working hours, minimum block length, and daily limits are respected
Portfolio allocation	Shared free time is allocated once across all active commitments in deterministic deadline order
Deterministic scheduling	Required work is split into valid app-owned blocks before the deadline without double-counting capacity
Verified progress	Elapsed time never reduces remaining effort without explicit progress or completion evidence
Continuous reconciliation	A moved or newly added event triggers a new state comparison

Capability	P0 acceptance condition
Stable repair	Only the minimum necessary future CommitmentOS blocks move
Stable external identity	A work block keeps one persisted Calendar event ID across plan revisions, repairs, retries, and executor restarts
User-edited blocks	A valid manual move is adopted; deletion requests a reschedule, pause, or progress decision instead of silently recreating the block
Risk	Portfolio allocation, shortfall, and still-unallocated buffer produce a reproducible result
Projection integrity	Stored aggregate and risk projections always identify their authoritative inputs and planner or projection version; reconciliation repairs any stale projection before acting
Audit	Every decision and action records its reason, policy, and outcome
Execution controls	The controlled user can pause automatic actions; queued and newly written outbox records remain held and cannot reach Calendar until resumed and revalidated
Completion	Explicit manual confirmation closes a commitment without fabricating verified minutes or reopening during reconciliation

Capability	P0 acceptance condition
Authentication	The live app is restricted to the controlled account; a separate seeded judge route is read-only
Cloud deployment	The full loop runs on Google Cloud and is visible in the demo

6.2 P0 hard gate

Do not begin P1 until the exact Gmail-only competition scenario runs successfully ten consecutive times with:

- No duplicate commitments
- No duplicate Calendar blocks
- No hard scheduling violations
- No capacity minute allocated to more than one active commitment
- No manual repair outside intentional effort/plan confirmation or an explicit user edit decision
- A complete audit record for every Calendar mutation
- Correct adoption of a valid manually moved app-owned block
- No automatic recreation of a user-deleted block before the user chooses reschedule, pause, or progress
- Safe recovery from replayed source events
- Safe continuation after a Cloud Run instance is recycled between input request and user response
- Safe continuation after the outbox writer or Calendar executor is recycled
- No new Calendar mutation while automatic actions are paused, including from tasks that were queued before the pause
- Safe resumption in which held actions are revalidated and stale actions are superseded rather than executed
- No divergence between authoritative work-block evidence and stored commitment projections after replay or concurrent check-in tests

6.3 P1 — commitment semantics and a second source

Only after the P0 gate:

- Sent-email completion candidates

- Canvas assignment adapter using a controlled test account or personal access token
- Dependency-edge behavior
- External owner state
- Source deadline changes
- Dependency-driven blocked state
- Contextual follow-up drafts with explicit approval
- Canvas submission evidence

6.4 P2 — breadth and learning

- Google Drive completion evidence
- Adaptive effort estimation from historical outcomes
- Outlook and multiple-account support
- Shared commitments and team views
- Rich investigation interface
- What-if planning
- Voice or multimodal approvals
- Weekly reliability reports
- Production multi-user OAuth onboarding and token storage

7. Locked Technical Stack

7.1 Model

- Gemini 3.5 Flash
- Stable model ID: `gemini-3.5-flash`
- Accessed through the Gemini API for the hackathon implementation
- Structured outputs enabled for commitment extraction
- Low thinking level for routine extraction; increase only for explicitly ambiguous cases if evaluation shows a material benefit
- No direct Calendar or Gmail write tools exposed to the extraction agent

The model version must be centralized in configuration and written into every model-backed audit event.

7.2 Agent framework

- Google ADK 2.x for Python
- Graph workflow for explicit nodes, branching, and typed state inside bounded invocations

- Deterministic function nodes for validation, portfolio risk, scheduling, and policy
- Gemini-backed agent node only where human-language interpretation is required
- Application-level input requests persisted in Firestore rather than an invocation waiting in memory
- No external Calendar mutation inside an ADK graph node

ADK session history may be retained for tracing, but it is not the authoritative commitment store. P0 does not require a persistent ADK `SessionService`. If native ADK `RequestInput` is added later, the implementation must first configure a persistent session service, resumability, invocation ID storage, and replay-safe tools.

7.3 Application stack

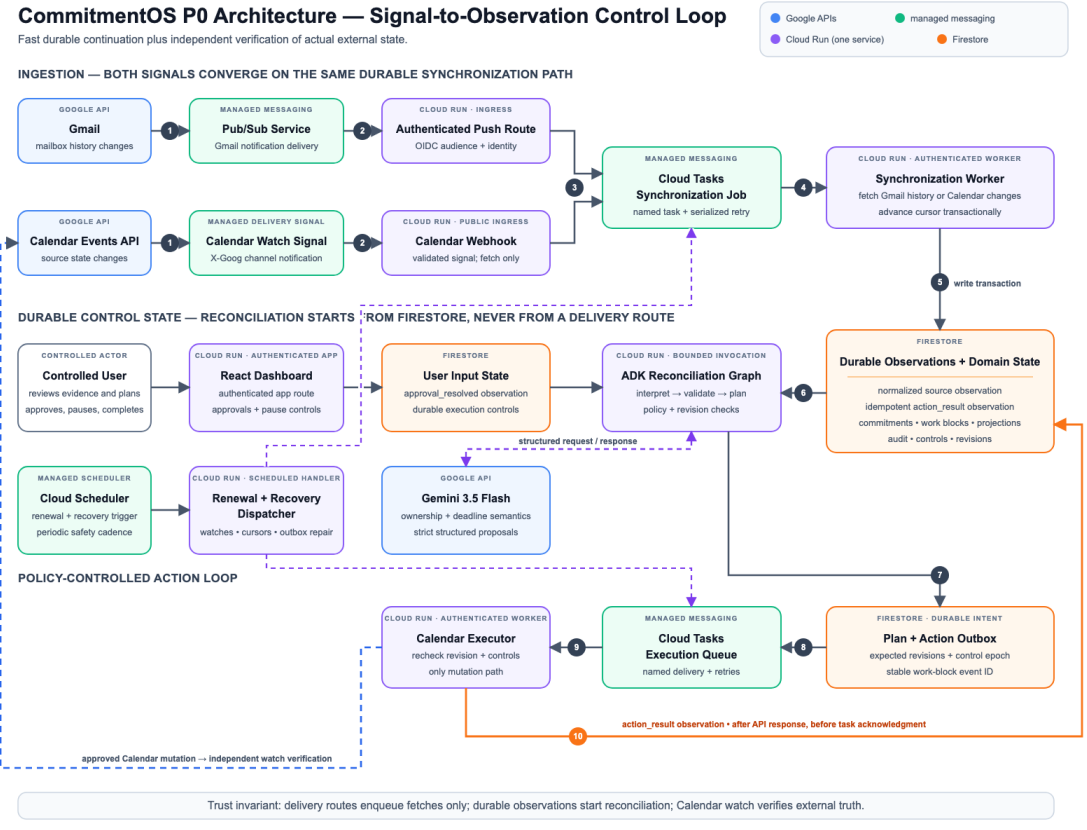
- One Python/FastAPI service on Cloud Run
- React/TypeScript dashboard compiled to static assets and served by the same Cloud Run service, avoiding a second deployment and cross-origin authentication path
- Cloud Run invocation allowed at the IAM edge because Google Calendar must reach a public HTTPS webhook; FastAPI applies route-specific authentication and authorization before any business operation
- Firestore for authoritative application state, observations, approvals, audit events, synchronization cursors, per-user processing leases, and durable execution controls
- Pub/Sub for Gmail notifications and normalized event delivery
- Cloud Tasks for named idempotent retries, delayed execution, and serialized per-user synchronization
- Cloud Scheduler for watch renewal, cursor catch-up, and periodic safety reconciliation
- Secret Manager for the Gemini API key, OAuth client credentials, and P0 test-user refresh token
- Cloud Logging for operational evidence and debugging

Production token storage is a post-hackathon security design. The P0 Secret Manager approach is deliberately optimized for a controlled test account, not claimed as a multi-tenant token vault.

7.4 P0 architecture boundary

The single Cloud Run service contains four explicitly separated route classes:

1. **Authenticated application routes:** dashboard data, approvals, completion, monitoring controls, and action-pause controls; protected by the controlled-user session and CSRF validation.
2. **Google-authenticated delivery routes:** Pub/Sub push and Cloud Tasks handlers; protected by application-side validation of Google-signed OIDC tokens, expected audience, and service identity.
3. **Calendar webhook route:** publicly reachable because Calendar push does not attach the configured Pub/Sub or Cloud Tasks OIDC identity; protected by a high-entropy channel token, channel ID, resource ID, expected Calendar mapping, strict method and body rules, and rate limiting. A valid signal can only enqueue a Calendar fetch.
4. **Read-only demo routes:** seeded data only, with no live mailbox content, approval endpoints, execution controls, credentials, or mutation capability.



CommitmentOS P0 architecture: scheduler-driven recovery, durable observations and domain state, fast action-result continuation, and independent Calendar-watch verification

The diagram is an implementation baseline, not a claim that the system already exists. Update it only when a deployed architectural boundary changes.

8. ADK Reconciliation Workflow

The root ADK workflow should expose the product’s real control plane in code.

8.1 Workflow nodes

Node	Type	Responsibility
`load_observation`	Function	Load normalized source input and synchronization metadata
`interpret_commitment`	Gemini agent	Extract or update commitment semantics using a strict schema
`validate_interpretation`	Function	Reject unsafe, incomplete, or impossible model output
`resolve_commitment_identity`	Function plus bounded Gemini classification	Choose create, update, supersede, ignore, or ambiguous against thread-linked commitments
`upsert_evidence`	Function	Deduplicate evidence and link it to the resolved commitment candidate
`record_effort_input_required`	Function	Persist the request and terminate the current run safely
`load_reconciliation_state`	Function	Load all active commitments, work blocks, authoritative facts, projection provenance, policy, execution controls, Calendar facts, and revisions for the planning horizon

Node	Type	Responsibility
`calculate_portfolio_feasibility`	Function	Allocate shared free capacity once across active commitments and compute per-commitment slack and risk
`produce_stable_plan`	Function	Generate a constraint-safe portfolio plan or minimal repair
`apply_policy`	Function	Decide whether the action is automatic, requires approval, or is forbidden
`record_action_approval_required`	Function	Persist exceptional-change approval and terminate safely
`write_action_outbox`	Function	Transactionally persist revision-aware intended actions, stable external IDs, control epochs, and dispatch metadata before external mutation
`finalize_reconciliation_run`	Function	Record the bounded run result and terminate without waiting for Calendar I/O
`verify_completion`	Function	Validate explicit user completion evidence and apply the terminal-state invariant

Calendar execution is deliberately absent from this graph. A separate authenticated Cloud Task handler owns external mutation and result recording, as specified in Section 15.

8.2 Agent boundary

Gemini may:

- Interpret commitment language
- Classify ownership
- Propose one conservative effort estimate for user confirmation
- Explain a deterministic decision in plain language

Gemini may not:

- Directly mutate Calendar
- Select an action that violates scheduling constraints
- Override an approval requirement
- Mark arbitrary commitments complete without valid evidence
- Read secrets or authentication material
- Treat source-email instructions as system or tool instructions

8.3 Durable input and continuation contract

When effort confirmation or action approval is required:

1. Write an `approval` record containing the commitment revision, proposed change, policy reason, continuation type, and expiration.
2. Write an activity event and finish the ADK invocation without side effects.
3. Render the approval from Firestore in the dashboard.
4. Accept the user's structured response through an authenticated FastAPI endpoint with CSRF protection.
5. Store the response once using the approval ID as the idempotency key.
6. Publish an `approval_resolved` source observation.
7. Start a new bounded graph run that re-reads the current commitment revision before planning or acting.

If the commitment changed while approval was pending, the old approval becomes `superseded` and the system recalculates instead of applying stale intent.

8.4 External action continuation contract

When the policy permits a Calendar mutation:

1. In a Firestore transaction, write one or more `action_outbox` records containing expected revisions, the persisted work-block external ID, the current execution-control epoch, before state, and action idempotency keys.
2. Attempt to create named Cloud Tasks for those records; a periodic dispatcher repairs the write-before-enqueue gap.
3. End the ADK reconciliation run without calling Calendar.
4. The Cloud Task handler authenticates the task request, reloads the current revisions and durable execution controls, and holds paused, stale, or completed actions.
5. Immediately before Calendar I/O, the handler rechecks that automatic actions remain enabled and that the control epoch and expected revisions still match.
6. The handler inserts, updates, adopts, or cancels only owned Calendar events using the work block's already-persisted external ID.
7. In one Firestore transaction, the handler records the terminal, held, or retry state and creates an idempotent `action_result` observation when execution reaches a terminal outcome.
8. It dispatches a named reconciliation task; the periodic dispatcher repairs the observation-before-enqueue gap.
9. A new bounded ADK run consumes that observation and verifies desired versus actual state.

`record_outcome` belongs to the Cloud Task executor, not to the reconciliation graph. This separation is the single authoritative external-action path.

9. Source Integration Semantics

9.1 Gmail ingestion

Implementation sequence:

1. Register `users.watch` for the controlled Gmail account and publish to Pub/Sub.
2. Persist the returned mailbox `historyId` and watch expiration.
3. On Pub/Sub delivery, decode the account and new history ID.
4. Durably enqueue a named per-user synchronization task and acknowledge the Pub/Sub delivery.

5. Acquire a short Firestore lease so only one Gmail cursor worker runs for that account.
6. Call `history.list` from the last committed cursor through the newest observed history ID.
7. Fetch only newly relevant messages or thread changes.
8. Normalize messages into immutable source observations.
9. Commit observations and the new history cursor transactionally, then release the lease.

Operational requirements:

- Renew the Gmail watch daily through Cloud Scheduler.
- Filter normalized changes to relevant Inbox and Sent activity.
- Treat Pub/Sub delivery as at-least-once.
- Do not use Pub/Sub message ordering as a cursor correctness mechanism.
- Use a per-user queue or lease so concurrent notifications cannot race the same history cursor.
- Fall back to periodic `history.list` catch-up when no notification arrives.
- Handle an invalid or expired history cursor with a bounded resynchronization.
- Prevent notifications caused by the application from creating loops.

9.2 Calendar observation

Implementation sequence:

1. Establish an Events watch channel for the selected Calendar.
2. Receive the HTTPS notification at the dedicated public Calendar webhook route on Cloud Run. Calendar does not use the Pub/Sub or Cloud Tasks OIDC delivery identity for this call.
3. Before any enqueue, validate the high-entropy channel token, channel ID, resource ID, expected Calendar mapping, allowed method, empty-body expectation, and current or overlap-renewal channel status.
4. Rate-limit the route, durably enqueue a named Calendar synchronization task, and return success. The webhook itself cannot read Calendar data, start ADK, or mutate state beyond the validated enqueue record.
5. Acquire a per-calendar synchronization lease.
6. Use the persisted Calendar sync token to fetch actual event changes.
7. Normalize the changes into observations.
8. Trigger reconciliation only for affected active commitments or capacity windows.

9. Persist the next sync token after a successful incremental sync and release the lease.

Operational requirements:

- Notifications contain change signals, not event bodies; always fetch the changed resources.
- Treat the channel token as a secret capability: generate at least 256 bits of entropy, store only in protected configuration or encrypted state, never include OAuth material, and redact it from logs.
- A spoofed or replayed valid-looking signal can at most cause an authenticated, rate-limited source fetch; it can never directly invoke reconciliation or Calendar mutation.
- Replace expiring Calendar channels with new unique channel IDs.
- On HTTP `410`, discard the invalid sync cursor and perform a full bounded resync.
- Use a periodic reconciliation pass as a safety net.
- Store the Calendar event `etag` and use revision guards against concurrent updates.

9.3 Calendar ownership

Every CommitmentOS-created event must include private extended properties:

```
{
  "managed_by": "commitmentos",
  "commitment_id": "commitment_123",
  "work_block_id": "block_456",
  "plan_revision": "7"
}
```

The executor may create, move, or cancel only events containing valid CommitmentOS ownership metadata. It must never mutate unrelated user events.

9.4 Calendar create idempotency

For every new work block, derive one stable Calendar event ID from an application namespace, the Calendar ID, and the immutable `work_block_id` using a base32hex-safe hash. Persist that external ID on the work block in the same transaction that first creates the work block. Supply it to Calendar during `events.insert` rather than relying on a server-generated ID.

The persisted Calendar event ID must never contain `plan_revision`, action type, retry number, proposed start time, or any other mutable value. Plan revisions belong in the action idempotency key and event extended properties; they can change how the same event is updated but cannot change the event's identity. A genuinely new work block receives a new `work_block_id` and therefore a new external ID.

If execution succeeds but the worker crashes before recording the response, a retry must fetch the persisted deterministic event ID or treat an already-existing response as success after verifying its private ownership properties. Extended properties remain the ownership and audit mechanism; stable work-block identity closes the create-before-record crash window without creating a second event after a later plan revision.

9.5 User edits to CommitmentOS blocks

An owned event whose actual state differs from the last successful outbox result, with no matching pending action, is treated as a user edit rather than an ordinary scheduling conflict.

- **Valid manual move:** Adopt the new time as desired state, increment the plan revision, and reconcile the remaining portfolio around it.
- **Invalid manual move:** Preserve the observed event temporarily and request a choice because it violates a hard constraint or overlaps another event.
- **User deletion:** Mark the block `user_deleted` and request one structured decision: reschedule the unfinished minutes, record completed minutes, or pause the commitment. Do not silently recreate it.
- **Unrelated event creates an overlap:** Treat this as environmental disruption and automatically repair only the affected future owned blocks within policy.
- **Application-generated change:** Match it to the outbox action and record the result without starting a duplicate repair loop.

The activity timeline must state whether CommitmentOS adopted user intent, requested clarification, or repaired an external disruption.

9.6 Commitment identity across messages

Before creating a commitment from a new Gmail observation:

1. Load active, candidate, dismissed, and recently completed commitments linked to the Gmail thread.
2. Build a bounded comparison set containing IDs, ownership, normalized outcome, beneficiary, deadline, and supporting evidence references.

3. Ask the interpretation node for structured semantics and a proposed identity operation.
4. Validate the target exists, belongs to the same user, and is compatible with deterministic ownership and thread rules.
5. Apply `create`, `update_existing`, `supersede`, or `ignore`; route `ambiguous` to confirmation.
6. Record the candidate set, proposed operation, final operation, and reason in the audit timeline.

A dismissed source span must not reappear unchanged after later thread activity. Multiple distinct commitments in one message receive stable source-span keys so replay remains idempotent.

9.7 Deadline normalization

Relative expressions such as “Friday,” “tomorrow,” or “end of day” are interpreted using the message timestamp, the controlled user’s IANA timezone, and a configured working-day end. The normalized value, source expression, timezone, rule version, and confidence are stored together. A date-only deadline defaults to the configured working-day end; low-confidence or conflicting interpretations require confirmation before planning.

10. Domain and Persistence Model

10.1 Commitment

```
{
  "commitment_id": "commitment_123",
  "user_id": "user_1",
  "revision": 7,
  "source_thread_id": "thread_123",
  "semantic_fingerprint": "my_commitment:send-revised-
proposal:professor-chen",
  "title": "Send revised proposal",
  "description": "Send the revised proposal to Professor Chen",
  "ownership_type": "my_commitment",
  "owner": { "type": "user", "display_name": "Me" },
  "beneficiary": { "display_name": "Professor Chen" },
  "deadline": {
    "value": "2026-08-14T17:00:00-07:00",
    "timezone": "America/Los_Angeles",
    "confidence": 0.93,
```

```
    "evidence_id": "evidence_123"
  },
  "effort": {
    "proposed_minutes": 180,
    "confidence": 0.58,
    "confirmed_minutes": 180,
    "confirmed_at": "2026-08-10T18:00:00Z"
  },
  "lifecycle_status": "active",
  "completion_evidence_id": null,
  "completed_at": null,
  "plan_revision": 3,
  "projection": {
    "verified_completed_minutes": 0,
    "remaining_minutes": 180,
    "risk_level": "on_track",
    "blocking_status": "clear",
    "source_commitment_revision": 7,
    "source_work_block_revision_hash": "sha256:...",
    "planner_run_id": "planner_run_789",
    "calculator_version": "portfolio-risk-v1",
    "computed_at": "2026-08-10T18:01:00Z"
  },
  "policy_profile": "default_personal",
  "last_reconciled_at": "2026-08-10T18:01:00Z",
  "created_at": "2026-08-10T17:59:00Z",
  "updated_at": "2026-08-10T18:01:00Z"
}
```

`semantic_fingerprint` is a matching aid, not an autonomous merge authority. Thread-linked existing commitments and evidence are always checked before creation. The resolver emits one of `create`, `update_existing`, `supersede`, `ignore`, or `ambiguous`; ambiguous identity never silently merges records.

The authoritative commitment facts are source evidence, normalized ownership and deadline, confirmed effort, lifecycle status, completion evidence, policy, and revisions. `verified_completed_minutes`, `remaining_minutes`, `risk_level`, and `blocking_status` are read projections. They must be stored together with the exact commitment revision, work-block revision hash, planner run, calculator version, and computation time that produced them. They are never accepted as independent mutation inputs.

10.2 Work block

```
{
  "work_block_id": "block_456",
  "commitment_id": "commitment_123",
  "calendar_id": "primary",
  "calendar_event_id": "6k9m2...base32hex",
  "duration_minutes": 60,
  "execution_state": "planned",
  "scheduled_start": "2026-08-11T09:00:00-07:00",
  "scheduled_end": "2026-08-11T10:00:00-07:00",
  "verified_minutes": 0,
  "completion_evidence_id": null,
  "user_edit_state": "none",
  "etag": "calendar-etag",
  "plan_revision": 3
}
```

Only `verified_minutes` contributes to commitment progress. Moving or deleting a Calendar event changes the plan, not the amount of completed work.

`verified_minutes`, its actor, timestamp, and evidence reference are authoritative progress facts. `calendar_event_id` is assigned once from the immutable work-block identity and persisted before any outbox record is created. Scheduled times describe desired domain state; the separately observed Calendar event, its `etag`, and the last successful action result describe actual external state.

10.3 Supporting collections

Collection	Required fields or purpose
`source_observations`	Source, external ID, external version, observed time, payload hash, idempotency key
`evidence`	Commitment ID, source reference, minimal excerpt, confidence, model version, schema version
`work_blocks`	Commitment ID, deterministic Calendar event ID, duration, execution state, verified minutes, evidence, etag, plan revision
`dependency_edges`	P1 source commitment, target commitment, type, owner, status
`approvals`	Request type, commitment revision, payload, continuation type, policy reason, expiration, decision, actor, timestamps
`action_outbox`	Intended mutation, expected commitment and plan revisions, persisted external ID, action idempotency key, expected control epoch, dispatch status, execution status, attempts, before/after, error
`activity_events`	Human-readable and machine-readable audit timeline
`sync_cursors`	Gmail history ID, Calendar sync token, channel IDs, expirations, last success
`processing_leases`	User/source key, lease owner, acquired time, expiration, last completed cursor

Collection	Required fields or purpose
`web_sessions`	Opaque session ID hash, allowlisted user ID, CSRF secret, created time, expiration, revoked time
`planner_runs`	Input revisions, planning horizon, deterministic ordering, capacity allocation, risk results, and planner version
`system_controls`	Controlled user, observation mode, automatic-action mode, monotonically increasing control epoch, actor, reason, and timestamps

10.4 Authoritative facts and projections

The persistence model follows four rules:

1. **Facts are append-backed:** source observations, evidence, approval decisions, work-block check-ins, completion evidence, and external action results are immutable or revisioned business facts.
2. **Domain records are transactional:** commitment and work-block revisions change only in Firestore transactions that also record their triggering observation or evidence reference.
3. **Projections are replaceable:** portfolio allocations, remaining minutes, risk, and blocking state may be cached for reads, but every projection carries input revisions and a calculation version. A mismatched projection is stale and must be recomputed before policy or execution.
4. **Completion is authoritative:** completion evidence closes the commitment. A projection may set remaining work to zero for display, but it must never manufacture verified minutes or reopen a completed commitment.

The reconciliation loader validates projection provenance. If the stored commitment revision or work-block revision hash no longer matches, it records `projection_stale`, recalculates, and only then proceeds to policy. An executor rejects any action whose expected projection or plan inputs no longer match current facts.

10.5 Idempotency keys

Suggested forms:

```
gmail:{user_id}:{message_id}:{extractor_schema_version}
calendar:{calendar_id}:{event_id}:{etag}
action:{commitment_id}:{plan_revision}:{action_type}:{target_id}
approval:{approval_id}:{decision}
```

Calendar event identity is a separate derivation, not an action idempotency key:

```
external_event_id = base32hex(sha256("commitmentos:v1" + calendar_id +
work_block_id))
```

The external ID is persisted once. Retries and later plan revisions continue using that value, while each intended mutation receives its own revision-aware action idempotency key.

Bidirectional dependency arrays must not be stored inside a commitment as the primary representation. P1 dependencies belong in an edge collection to avoid stale denormalized graphs.

11. Deterministic Portfolio Risk Engine

11.1 P0 portfolio calculation

P0 must never give the same free Calendar minute to more than one commitment. Risk is therefore calculated only after one global hypothetical plan is built for all active commitments in the planning horizon.

For each incomplete commitment:

```
verified_completed_minutes = sum(work_block.verified_minutes)
remaining_minutes = max(confirmed_effort - verified_completed_minutes,
0)
preserved_reserved_minutes = sum(valid future owned blocks retained in
the plan)
allocation_deficit = max(remaining_minutes -
preserved_reserved_minutes, 0)
```

The planner then:

1. Generates one shared set of eligible free slots after removing unrelated busy events and all preserved owned blocks.

2. Sorts commitments by confirmed deadline, explicit user priority, creation time, and commitment ID as a final stable tie-breaker.
3. Allocates each free slot at most once until every commitment's deficit is filled or no eligible slot remains.
4. Records `allocated_work_minutes`, `shortfall_minutes`, projected finish, and the still-unallocated portfolio capacity before each deadline.

For each commitment:

```
allocated_work_minutes = preserved_reserved_minutes +
newly_allocated_minutes
shortfall_minutes = max(remaining_minutes - allocated_work_minutes, 0)
portfolio_slack_minutes = eligible minutes still unallocated before
this deadline
slack_ratio = portfolio_slack_minutes / max(remaining_minutes, 30)
```

Initial thresholds:

```
lifecycle_status == completed                  → closed, no active
risk
deadline passed and incomplete                 → overdue
effort not confirmed                           → unknown
shortfall_minutes > 0                          → critical
shortfall_minutes == 0 and slack_ratio < 0.25 → at_risk
slack_ratio ≥ 0.25                             → on_track
```

The same unused minute may be reported as portfolio buffer for commitments with overlapping horizons, but it is never allocated as work twice. The audit event records the commitment ordering, input revisions, confirmed effort, verified progress, preserved blocks, new allocation, shortfall, unused portfolio capacity, threshold version, and previous/new risk.

After calculation, the planner publishes one `planner_run` plus the affected commitment projections in a Firestore transaction that first verifies the complete expected revision set. If any fact changed, the transaction writes no projection or outbox action; the run is marked stale and a new calculation begins from current facts.

11.2 Progress evidence

P0 accepts two explicit progress paths:

- **Work-block check-in:** The user records minutes actually completed after a block. The update is bounded by the block duration and stored with actor and timestamp.
- **Commitment completion:** Manual confirmation writes completion evidence, sets `completed_at`, and closes the commitment while retaining the original estimate and verified-minute history.

The system must never infer completed minutes only because a block's end time has passed. It must not fabricate verified minutes to make a completed commitment match its estimate. If a block is partially completed, the verified portion is retained and the remaining portion is replanned. Sent-email completion candidates are P1 and, even then, require confirmation unless evaluation supports a narrowly defined deterministic rule.

11.3 P0 limitations

P0 deliberately excludes probabilistic productivity forecasts, adaptive personal models, and dependency-adjusted capacity. It uses one controlled user and one primary Calendar, but supports multiple active commitments for that user. The goal is a transparent portfolio calculation a judge can reproduce from the screen.

12. Deterministic Scheduling and Stable Repair

12.1 Hard constraints

- Existing non-CommitmentOS busy events
- Commitment deadline and timezone
- User working hours
- Minimum session length
- Maximum work-block length
- Daily focus-work limit
- No overlap
- No scheduling in the past
- Only app-owned blocks may be mutated
- Each eligible time interval allocated to at most one commitment

12.2 Soft preferences

- Preferred focus periods
- Earlier completion and deadline buffer
- Fewer fragmented sessions

- Balanced daily load
- Avoiding back-to-back work when alternatives exist

12.3 Portfolio planning algorithm

P0 uses a deterministic greedy planner:

1. Load all active commitments and their expected revisions for the horizon.
2. Retrieve busy intervals and classify valid owned blocks, user-edited owned blocks, unrelated events, and pending outbox mutations.
3. Preserve valid approved blocks unless a hard constraint or explicit user decision requires a change.
4. Generate one shared candidate-slot pool at a fixed interval such as 15 minutes.
5. Remove candidates violating hard constraints and score the remainder with stable documented preferences.
6. Sort commitment deficits by deadline, explicit priority, creation time, and commitment ID.
7. Allocate each candidate slot once across the portfolio, retaining stable existing blocks before adding or moving work.
8. Produce a versioned portfolio plan, per-commitment risk results, and intended mutations.
9. For a first plan, request confirmation. For an allowed repair, write actions to the outbox; never call Calendar from the planner.

An optimization solver is not required for P0.

12.4 Repair objective

When actual Calendar state changes, apply this priority order:

1. Never violate a hard constraint.
2. Preserve completed and currently active work blocks.
3. Adopt a valid explicit user move before optimizing other blocks.
4. Preserve all unaffected future blocks across every commitment.
5. Move the smallest possible number of affected future blocks.
6. Minimize total displacement from the approved portfolio plan.
7. Restore portfolio feasibility and deadline buffer where possible.
8. Escalate instead of pretending success when no feasible repair exists.

The demo explanation should say exactly which block moved, why it moved, what remained unchanged, and how risk changed.

13. Autonomy, Safety, and Privacy

13.1 P0 autonomy policy

Action	Policy
Detect and record a possible commitment	Automatic
Show a candidate in the dashboard	Automatic
Infer an uncertain deadline or owner	Require confirmation
Create the first Calendar plan	Require effort and plan confirmation
Repair app-owned blocks within approved preferences	Automatic with notification and undo
Adopt a valid manual move of an app-owned block	Automatic with explanation
Respond to a user-deleted app-owned block	Require reschedule, progress, or pause decision; never silently recreate
Make extensive changes or exceed daily limits	Require renewed approval
Modify a non-CommitmentOS event	Forbidden
Send an external email	Not in P0; always approval-gated in P1
Mark a commitment complete	Require explicit confirmation in P0
Pause or resume monitoring	Controlled-user action; effective for new source-processing work and recorded in the audit timeline
Pause or resume automatic Calendar actions	Controlled-user action; immediately increments the control epoch and holds all not-yet-started external mutations

Action	Policy
Run controlled-account cleanup	Developer-only authenticated operation in P0; self-service UI is deferred

“Extensive change” must be deterministic configuration, for example moving more than two blocks, shifting total work by more than one day, or scheduling outside preferred hours.

13.1.1 Durable execution controls

P0 stores one per-user `system_controls` record with `observation_mode`, `automatic_action_mode`, `control_epoch`, actor, reason, and timestamps.

- Pausing monitoring prevents new observation-processing tasks from starting. Already-normalized observations remain durable and are reconsidered after resume.
- Pausing automatic actions increments `control_epoch`, prevents the dispatcher from releasing pending outbox records, and causes queued executors to mark their records `held_by_control` before Calendar I/O.
- Every outbox record carries the control epoch observed when intent was written. Every executor checks the current mode and epoch when claiming the action and again immediately before the Calendar request.
- Resuming increments the epoch again and emits a `system_control_changed` observation. Held actions are never released blindly; reconciliation revalidates current commitment, plan, Calendar, and control revisions and writes fresh intent where necessary.
- A Calendar request that has already crossed the external-I/O boundary cannot be atomically canceled. The UI must expose `action_in_flight`, and the resulting Calendar observation must still be reconciled and audited.
- Undo remains a compensating reconciliation action. It is not a substitute for the action-pause control.

13.2 Prompt-injection boundary

Email and Canvas content are untrusted data. P0 must enforce:

- Source text is clearly delimited as data in model requests.
- The extraction agent has no mutation tools.
- Model output must satisfy a strict schema.

- Deterministic code validates dates, ranges, ownership enums, and confidence.
- Model-produced text can never bypass the policy node.
- Secrets, tokens, internal prompts, and unrelated emails are excluded from model context.
- Adversarial email fixtures are included in evaluation.
- Logs redact message bodies, OAuth material, and Gemini API credentials.

13.3 Data minimization

- Process the required email body transiently.
- Persist message and thread IDs, a short evidence excerpt, structured fields, and a content hash.
- Avoid storing complete mailbox contents.
- Provide separate monitoring-pause and automatic-action-pause controls in the P0 UI, with current status visible on every live-data view.
- Provide a documented authenticated developer cleanup command for the controlled account; public self-service disconnect and deletion UI is P1.
- Record monitoring, cleanup, access, and deletion events in the audit timeline.

13.4 OAuth plan

P0 uses a named Gmail/Calendar test account and the narrowest functional scopes:

- Gmail read access for commitment and Sent evidence
- Calendar events read/write access
- Basic identity scopes

Do not request Gmail send or modify scope in P0. Refresh tokens and watches can expire, so the UI must surface `reauth_required` instead of failing silently. Reconnect the recording account shortly before the final demo.

Phase 0 must test two OAuth publishing configurations with the real controlled account and exact requested scopes:

1. **External, In production, unverified personal-use configuration:** Keep application access restricted by the CommitmentOS allowlist. Confirm whether the controlled user can click through the unverified-app warning and retain a refresh token beyond the Testing-mode window while remaining under Google's unverified-user cap. Do not claim public verification or general-user readiness.

2. **External, Testing configuration:** Use the controlled account as an explicit test user and assume the Gmail/Calendar refresh token expires after seven days because the requested scopes exceed basic identity access.

Choose the In-production personal-use configuration only if the Phase 0 spike proves authorization, refresh, watch renewal, revocation, and account allowlisting end to end. Otherwise use Testing and execute the reconnection runbook. In both modes, refresh tokens and watches can expire or be revoked at any time, so `reauth_required` remains mandatory.

The implementation runbook must include:

- An end-to-end reauthorization test during the integration spike
- Scheduled developer reconnection during the 21-day build
- Reconnection immediately before golden-run testing and video recording
- No silent fallback to stale cached source data
- A seeded judge mode that demonstrates the interface without asking judges to connect Gmail
- The selected publishing status, unverified-user limitation, exact scopes, and observed refresh-token behavior

The hosted URL may expose seeded or read-only demonstration data. P0 does not promise public multi-user Gmail onboarding or OAuth verification. Never share the controlled Google account's credentials with judges.

13.5 Endpoint authentication

- Serve the React dashboard and FastAPI routes from the same Cloud Run origin.
- Configure the single Cloud Run service to permit unauthenticated invocation at the IAM edge because Calendar must call a public HTTPS webhook. Treat Cloud Run IAM admission as transport reachability, not application authorization; every route enforces its own trust contract before business logic.
- Protect `/app` with Google OAuth authorization-code login, validate `state` and nonce, and allow only the controlled test-account identity.
- Store an opaque server-side session in Firestore; issue only a `Secure`, `HttpOnly`, `SameSite=Lax` session cookie and require CSRF tokens for mutations.
- Expose `/demo` as seeded, read-only judge mode. It must not contain live mailbox data, external credentials, approval endpoints, or mutation capability.
- Require and validate Google-signed OIDC tokens with the expected audience and service identity on Pub/Sub push and Cloud Tasks endpoints.

- Isolate the Calendar webhook to one exact path and method. Validate its high-entropy channel token with constant-time comparison plus channel ID, resource ID, expected Calendar mapping, and channel status before enqueueing work; Calendar webhook authentication must never be described as OIDC.
- Keep webhook handlers fast: validate, durably enqueue, and return. Business reconciliation belongs in Cloud Tasks workers.
- Rate-limit change-signal endpoints and ensure a spoofed notification can trigger only an authenticated source fetch, never a direct mutation.

14. Dashboard — P0 Only

P0 has three primary views.

14.1 Today

- Outcome strip: commitments kept feasible, minutes reserved, conflicts repaired, blocks preserved, and manual reschedules avoided
- Today's CommitmentOS work blocks
- Newly detected candidates
- At-risk or critical commitments
- Pending confirmation, approval, or deleted-block decision
- Live monitoring and automatic-action status, including held or in-flight action counts
- Controlled-user controls to pause or resume monitoring and automatic actions, with an explicit confirmation for action resume

14.2 Commitments

- Lifecycle, ownership, deadline, risk, and remaining effort
- Portfolio allocation, projected finish, shortfall, and shared buffer
- Source evidence and confidence
- Scheduled work blocks
- Work-block check-in and verified minutes
- Initial confirmation and pause/dismiss controls
- Manual completion control

14.3 Activity

- Observation received
- Interpretation created or rejected
- Confirmation recorded

- Risk before and after
- Scheduling proposal
- Portfolio allocation and stable ordering
- Policy decision
- Outbox write and Cloud Task dispatch
- Calendar execution result
- User move adopted or deletion decision requested
- Retry or failure
- Completion evidence
- Monitoring or action-control change, control epoch, actor, held actions, and resume revalidation result

The initial release does not need separate Calendar Plan or Dependencies pages. Calendar details belong inside a commitment. Dependencies become a P1 view only after the behavior exists.

15. Reliability and Recovery

15.1 Delivery model

Assume at-least-once delivery everywhere. Exactly-once product outcomes are approximated through action idempotency keys, stable Calendar IDs derived from immutable work-block identity, revision checks, source cursors, leases, and owned-event lookup—not by claiming the infrastructure or ADK invokes code only once. Every duplicate delivery must converge on the same domain and Calendar state.

15.2 Action outbox

The action outbox and its Cloud Task executor are the only path to external Calendar mutation:

1. Transactionally write the intended action, policy result, expected commitment and portfolio-plan revisions, action idempotency key, persisted work-block external ID, and expected execution-control epoch.
2. After commit, dispatch a named Cloud Task only when automatic actions are enabled. Otherwise mark the record `held_by_control` without losing intent.
3. Authenticate the Cloud Task request and re-read all expected revisions, projection provenance, automatic-action mode, and control epoch before execution.

4. Skip or hold stale, paused, superseded, or already-completed actions and durably record the decision.
5. For a create, first check the work block's persisted external event ID; then insert or adopt the existing event only after verifying its private ownership properties.
6. Recheck automatic-action mode and control epoch immediately before external I/O, then execute the Calendar mutation with etag or revision guards where supported.
7. Transactionally record the external ID, etag, outcome, new state, and one idempotent `action_result` observation before returning success to Cloud Tasks.
8. Dispatch a named reconciliation task for that observation; the periodic dispatcher repairs any observation-before-enqueue gap.
9. Match the later Calendar watch observation to the completed action so it verifies state without creating a duplicate repair loop.

A periodic dispatcher repairs pending outbox records that were written but not enqueued, but it cannot release work while automatic actions are paused. After resume, reconciliation first revalidates held intent and supersedes stale records. Retry exhaustion produces a visible `calendar_action_failed` state and never silently marks the desired plan as actual.

15.3 Concurrency

- Reconciliation operates on an expected commitment revision.
- Portfolio planning operates on the expected revisions of every active commitment in the horizon.
- Firestore transactions protect revision and outbox updates.
- Projection publication verifies the full input revision set; mismatched aggregates or risk fields are recalculated before policy.
- Execution-control updates increment a monotonic epoch. An action from an older epoch cannot mutate Calendar without fresh reconciliation.
- Gmail synchronization is serialized per user with a lease or queue concurrency of one.
- Approval resolution is compare-and-set: only one decision can win, and stale commitment revisions supersede the request.
- Stale planners must discard their output and recalculate.
- A valid user move increments the affected plan revision before a new portfolio calculation; a deletion remains unresolved until a user decision observation arrives.

- Pub/Sub ordering is not treated as a correctness guarantee.
- The activity record preserves all failed and superseded attempts.

15.4 Failure states visible to the user

```
reauth_required
source_sync_delayed
model_output_rejected
calendar_action_failed
reconciliation_retrying
no_feasible_plan
approval_superseded
work_check_in_required
user_block_decision_required
portfolio_capacity_conflict
action_stale
projection_stale
automatic_actions_paused
action_held_by_control
action_in_flight
```

Failures must not be hidden behind an “On Track” state.

16. Evaluation and Definition of Done

16.1 Extraction evaluation

Create at least 30 labeled Gmail fixtures covering:

- My explicit promise
- Request to me
- Another person’s promise
- External dependency language
- No commitment
- Ambiguous deadline
- Multiple commitments in one message
- Thread updates and changed deadlines
- Replies that restate the same commitment
- A new distinct commitment in an existing thread
- Dismissed evidence resurfacing after a later reply
- Prompt-injection attempts

Report:

- Schema-valid output rate
- Ownership accuracy
- Deadline accuracy
- False-positive candidate rate
- Identity-operation accuracy and duplicate-commitment count
- Model latency
- Cost per processed message

Target at least 90% ownership and deadline accuracy on the curated set, while preserving the raw measured result in the submission.

16.2 Scheduler tests

Include cases for:

- Timezones and daylight-saving transitions
- Existing all-day and recurring events
- Insufficient capacity
- Minimum block length
- Daily work limits
- Conflict inserted after planning
- Existing owned blocks count once as capacity
- Two commitments competing for the same free slot receive distinct allocations
- Deterministic deadline and tie-break ordering produces the same portfolio plan on replay
- Other commitments' preserved blocks remain unavailable
- One commitment becoming urgent can move only policy-permitted future owned blocks
- Elapsed but unconfirmed blocks do not count as progress
- Partial verified progress replans only the remainder
- Manual completion remains terminal without fabricating verified minutes
- Completed block preservation
- Valid manual block move is adopted
- Deleted block is not recreated before a user decision
- Minimal-change repair

Hard-constraint violations must equal zero.

16.3 Reconciliation tests

- Replay one Gmail notification repeatedly: one commitment only.
- Deliver overlapping Gmail history notifications concurrently: cursor advances once with no lost observations.
- Replay one Calendar notification repeatedly: one repair only.
- Send Calendar webhook requests with missing, incorrect, and stale channel credentials: each is rejected without a source fetch, reconciliation run, or mutation.
- Send one valid Calendar change signal: it can enqueue only the authenticated Calendar synchronization path and cannot pass event data or mutation intent through the webhook.
- Crash after writing an outbox record but before creating its Cloud Task: the periodic dispatcher recovers it.
- Interrupt Calendar create after the external insert but before outcome storage: deterministic event ID yields one Calendar outcome only.
- Increment `plan_revision` and replay a create or repair for the same work block: the persisted Calendar event ID remains unchanged and no second event appears.
- Replay an `action_result` observation and the matching Calendar watch notification: no second mutation.
- Recycle the Cloud Run instance after an input request: the later user response continues safely from Firestore.
- Resolve the same approval twice: one decision wins and one continuation observation exists.
- Invalidate a Calendar sync token: bounded full resync without state corruption.
- Let a Gmail watch expire in a fixture: renewal/catch-up path recovers.
- Move an owned block manually to a valid slot: the new time is adopted and preserved.
- Delete an owned block manually: no recreation occurs until the user chooses reschedule, progress, or pause.
- Complete a commitment with fewer verified minutes than estimated: later reconciliation keeps it completed.
- Corrupt or age a stored remaining-work or risk projection: the loader detects mismatched provenance, recalculates from authoritative facts, and blocks stale action execution.

- Queue a Calendar action, pause automatic actions before the executor runs, and deliver the task repeatedly: no Calendar mutation occurs and one held state is visible.
- Resume after changing the underlying commitment or Calendar: the held action is superseded and current state is replanned before any external mutation.
- Pause while an action is already in flight: the eventual result is observed and audited, and the UI never claims that the in-flight request was canceled.
- Serve seeded judge mode: live-data and mutation endpoints remain inaccessible.
- Run the golden demo scenario ten consecutive times.

16.4 Competition acceptance metrics

P0 is complete only when:

- Ten consecutive golden-path runs succeed.
- Conflict-to-repaired-plan latency is under 60 seconds operationally and under 15 seconds in the warmed demo environment.
- Duplicate commitment and work-block counts are zero under replay.
- No free Calendar interval is allocated to more than one active commitment.
- Every Calendar mutation has an audit event and idempotency key.
- A work block's Calendar event ID is stable across plan revisions and executor retries.
- Every Calendar mutation originates from an outbox record and authenticated task execution.
- No not-yet-started Calendar mutation executes while automatic actions are paused; held actions are revalidated after resume.
- No policy or executor decision consumes a projection whose recorded input revisions differ from authoritative facts.
- Invalid Calendar webhook credentials never enqueue a source fetch, and a valid webhook can trigger only Calendar synchronization.
- Every automatic repair touches only app-owned future blocks.
- Valid user moves are adopted and user deletions are not silently reversed.
- Uncertain commitments never create a first plan without confirmation.
- Invalid model output produces safe rejection rather than partial execution.
- Elapsed time alone never reduces remaining effort.
- Stale approvals are superseded instead of executed.

- Completion always has explicit user evidence, remains terminal on later reconciliation, and never fabricates verified minutes.
- The outcome strip and demo sentence are derived from stored audit events.

17. Implementation Roadmap

Phase 0 — integration risk spike, Days 1–2

- Freeze the golden scenario, autonomy policy, and narrow OAuth scopes.
- Create the cloud project, OAuth client, controlled test account, and minimal Cloud Run service.
- Configure the single Cloud Run service for public IAM-edge invocation, then prove the four application-level route contracts: controlled-user session, Pub/Sub/Tasks OIDC, Calendar channel-secret webhook, and mutation-disabled seeded demo.
- Prove invalid Calendar webhook credentials are rejected and a valid signal can enqueue only an authenticated Calendar fetch.
- Test External/In-production personal-use and External/Testing OAuth configurations with the exact scopes; record consent warnings, refresh behavior, revocation, allowlist enforcement, and the selected P0 mode.
- Prove same-origin login for the allowlisted account, authenticated Pub/Sub and Cloud Tasks requests, and a mutation-disabled seeded route.
- Prove Gmail watch → Pub/Sub → authenticated endpoint → durable Cloud Task.
- Prove Calendar watch → webhook → incremental fetch and channel renewal metadata.
- Prove stable work-block-derived Calendar event insert, lookup, update across a plan revision, and controlled cleanup without creating a second event.
- Prove Gemini 3.5 Flash structured output and one deployed ADK 2.x graph run.
- Complete one reauthorization cycle and observe the expected failure state for an invalid refresh token.

Gate: All external systems required by the golden path work from deployed Cloud Run code. No product UI is required yet.

Phase 1 — contracts and seeded vertical slice, Days 3–5

- Define Pydantic schemas, lifecycle transitions, identity operations, work-block states, authoritative facts, projection provenance, stable external identity, and Firestore collections.
- Implement immutable observations, activity events, approvals, revisions, system controls, and the action outbox.

- Implement a seeded observation that produces one commitment, one approval request, one approved plan, one outbox action, one authenticated task execution, and one `action_result` observation.
- Implement the automatic-action pause: queue a seeded outbox action, pause before execution, prove no Calendar mutation, then resume through revalidation.
- Prove an approval survives an application restart and continues through a new observation.
- Validate and revise the architecture diagram in Section 7.4 against the deployed trust boundaries before implementation diverges.

Gate: A seeded end-to-end run reaches real Calendar through a replay-safe outbox and remains recoverable across Cloud Run recycling.

Phase 2 — Gmail evidence and identity, Days 6–8

- Implement serialized per-user Gmail cursor processing, message fetch, normalization, and minimal evidence storage.
- Add daily watch renewal, catch-up, bounded cursor recovery, and loop prevention.
- Add Gemini structured extraction, deterministic validation, ownership classification, and identity operations.
- Build fixtures for restatements, deadline changes, multiple commitments, dismissals, and prompt injection, including the exact multi-message ownership and deadline-revision thread used in the competition demo.
- Add the candidate dashboard and source evidence view.

Gate: Real and replayed thread activity produces the correct commitment records with zero unintended duplicates.

Phase 3 — progress, capacity, and first plan, Days 9–11

- Add effort proposal and durable effort/plan confirmation.
- Implement work-block states, verified-minute check-in, the completion invariant, and active remaining-effort calculation.
- Read Calendar busy intervals and user preferences.
- Implement shared free-slot generation, deterministic portfolio ordering, single-allocation capacity math, and stable scoring.
- Create app-owned blocks using Calendar event IDs derived once from immutable work-block IDs and private extended properties; prove later plan revisions reuse those external IDs.

- Publish remaining-work and risk projections only with input revision hashes and planner-run provenance.
- Add safe undo through a new reconciliation event rather than blind state reversal.

Gate: Two active commitments produce a reproducible, constraint-safe portfolio plan with no shared minute allocated twice; elapsed time alone cannot alter progress.

Phase 4 — observation and minimal repair, Days 12–14

- Complete Calendar synchronization, renewal, token recovery, and affected-commitment routing.
- Implement deterministic portfolio risk calculation, minimal-change repair, valid user-move adoption, and deleted-block decision handling.
- Add revision guards, named Cloud Tasks, retry adoption, and notification-loop suppression.
- Enforce automatic-action mode and control-epoch checks in the dispatcher and immediately before Calendar I/O.
- Add periodic safety reconciliation and visible failure states.
- Tune the warmed demo path to repair within 15 seconds.

Gate: A newly inserted meeting automatically causes exactly one minimal repair with a complete before/after explanation.

Phase 5 — completion and hardening, Days 15–17

- Add manual completion and work-block check-ins; do not add Sent-message completion inference unless the hardening gate passes early.
- Add prompt-injection fixtures, redaction, final route-specific authentication, monitoring pause, automatic-action pause, and the documented controlled-account cleanup command.
- Exercise reauthorization, concurrent Gmail delivery, stale approvals, cursor recovery, projection corruption, pause-before-execution, stale-held-action resumption, and create-before-record crashes.
- Run the golden scenario ten consecutive times.

Gate: All Section 16 acceptance metrics pass with measured results preserved.

Phase 6 — competition delivery, Days 18–21

- Freeze product scope; do not start P1.
- Polish only Today, Commitments, and Activity.

- Finalize the architecture diagram, README, selected OAuth publishing status and limitations, seeded judge mode, and cloud spin-up instructions.
- Capture Cloud Run, Pub/Sub, Firestore, Cloud Tasks, and Logging evidence.
- Record the demo at least 48 hours before submission with one temporarily warm Cloud Run instance.
- Prepare a backup recording, seeded reset procedure, and OAuth reconnection checklist.
- Complete the Devpost write-up, measured outcomes, limitations, and technology list.
- Add optional public content and social promotion only after required submission assets are safe.

P1 work may begin only if Phase 5 passes early and competition materials are already complete.

18. Four-Minute Competition Demo

0:00–0:20 — problem and promise

Show a busy Calendar and say:

> Commitments are scattered through email, but a deadline alone does not reserve the time needed to deliver. CommitmentOS keeps the promise achievable until it is complete.

0:20–0:55 — evidence-backed detection

- Open the exact seeded multi-message Gmail thread used in evaluation:
  - Incoming request: “Could you take the proposal revision?”
  - User reply: “Yes—I’ll have it back before our Friday 4 p.m. review.”
  - Later update: “Could we bring the review forward to Thursday at 4?”
- Show one candidate—not duplicates—with `my_commitment` ownership, the normalized Thursday deadline, confidence, and highlighted evidence from the relevant messages.
- State that Gemini resolved who made the promise, connected the later deadline change to the existing commitment, and produced a structured proposal; deterministic code validated it before any plan existed.

0:55–1:25 — effort and initial plan

- Confirm the proposed three-hour effort.

- Show three valid work blocks appear around existing events and an already-active second commitment.
- Briefly point out that shared free time was allocated once across the portfolio.
- Point out that the first plan required confirmation.

1:25–1:50 — real-world disruption

- Add or reveal a meeting that displaces one CommitmentOS block.
- Return to the dashboard without pressing a “replan” button.
- In the recording environment, keep one Cloud Run instance warm and target visible reconciliation within 15 seconds.

1:50–2:30 — autonomous reconciliation

- Show the background observation arrive.
- Show risk change and the repaired Calendar.
- Highlight that only one affected future block moved while the other blocks—including the second commitment’s blocks—remained stable.
- Display the measured outcome sentence: one conflict repaired, one block moved, unaffected blocks preserved, feasibility restored.

2:30–3:00 — trust and auditability

- Open the Activity view.
- Show the source event, portfolio allocation, old/new risk, policy decision, outbox action, authenticated executor result, and actual Calendar observation.
- Point briefly to the visible “Automatic actions: On” control and explain that pausing it holds queued mutations for revalidation.
- Mention stable work-block event IDs, revision-aware idempotency, projection provenance, and app-owned event restrictions.

3:00–3:20 — completion

- Confirm completion explicitly.
- Show the completion evidence, verified progress history, and terminal closure without inventing extra work minutes.

3:20–3:50 — architecture and Google Cloud

- Show the architecture diagram.
- Identify Gemini 3.5 Flash, bounded ADK workflow runs, Firestore continuation state, deterministic portfolio planner, the separate outbox executor, Pub/Sub, Cloud Tasks, Cloud Scheduler, and Cloud Run.

- Briefly show Cloud Run or Cloud Logging execution evidence.

3:50–4:00 — closing line

> CommitmentOS is not an assistant that plans once. It keeps your promises feasible as reality changes—and proves when they are done.

Canvas, dependency follow-ups, and P1 features must not appear in the primary demo path. If P1 is stable, show it only as a short closing screenshot or mention it in the roadmap.

19. Submission Assets

Required competition assets:

- Hosted project URL if stable
- Concise problem and value proposition
- Feature and technology list
- Public or properly shared repository
- Reproducible local and cloud spin-up instructions
- Architecture diagram
- Approximately four-minute demo video
- Visible proof of Google Cloud deployment
- Findings, limitations, and learnings

Repository documentation should include:

- Exact model and ADK versions
- Required Google APIs and scopes
- Environment-variable and Secret Manager setup
- OAuth publishing-mode decision, controlled-user setup, unverified-app limitation, exact scopes, observed refresh behavior, and Testing-mode seven-day reconnection fallback
- Pub/Sub OIDC and Calendar public-webhook configuration as separate trust contracts
- Gmail per-user serialization and cursor recovery
- Firestore indexes
- Cloud Run deployment commands, including why public IAM-edge invocation is required and how every route is protected at the application layer
- Same-origin authentication, controlled-user allowlist, Pub/Sub/Tasks OIDC validation, Calendar channel-secret validation, and read-only seeded judge mode

- Portfolio allocation rules, authoritative-versus-projection state, stable work-block Calendar event ID derivation, action-result continuation, execution-control epochs, and outbox recovery
- Demo-data seeding and reset
- Evaluation commands and measured results
- Known restrictions around the selected OAuth mode, personal-use operation, and public production release

20. Features Explicitly Deferred

Do not add before P0 passes:

- Canvas
- Dependency graph UI or behavior
- Follow-up emails
- Gmail send scope
- Google Drive completion checks
- Sent-email completion inference
- Outlook
- Multi-account support
- Adaptive effort learning
- Voice approval
- Screenshot or PDF assignment understanding
- Shared team commitments
- What-if scheduling
- Weekly analytics
- Probabilistic effort ranges beyond one user-confirmed estimate
- Gemma bonus integration
- A broad chatbot
- Native long-lived ADK `RequestInput` sessions or a new SQL session store
- Public multi-user Gmail onboarding and OAuth verification
- Public self-service account disconnect and deletion UI
- Model Armor or enterprise platform components that do not improve the golden path

21. Final Success Definition

CommitmentOS succeeds when the following sequence runs repeatedly on real Gmail and Calendar data without hidden intervention:

> Detect → ground in evidence → resolve identity and ownership → confirm effort → allocate shared capacity once → write durable intent → honor execution controls → execute asynchronously → observe disruption → adopt user intent or minimally repair → explain and audit → verify completion.

The winning product is not the one with the most connectors. It is the one that makes this loop visible, reliable, safe, and unmistakably useful—even when events are replayed, an instance restarts, a plan revision changes, automatic actions are paused, or a person responds later.

22. Primary Technical References

- Hackathon brief supplied with the project: `All Things Agentic Hackathon.docx`
- Gemini 3.5 Flash model: https://ai.google.dev/gemini-api/docs/models/gemini-3.5-flash
- ADK graph workflows: https://adk.dev/graphs/
- ADK human input: https://adk.dev/graphs/human-input/
- ADK Cloud Run session persistence: https://adk.dev/deploy/cloud-run/
- ADK workflow resume behavior: https://adk.dev/runtime/resume/
- Gmail push notifications: https://developers.google.com/workspace/gmail/api/guides/push
- Calendar push notifications: https://developers.google.com/workspace/calendar/api/guides/push
- Calendar incremental synchronization: https://developers.google.com/workspace/calendar/api/guides/sync
- Calendar event insertion and client-supplied IDs: https://developers.google.com/workspace/calendar/api/v3/reference/events/insert
- Cloud Tasks HTTP targets and authenticated task delivery: https://cloud.google.com/tasks/docs/creating-http-target-tasks
- Cloud Run public-service authentication boundary: https://cloud.google.com/run/docs/authenticating/public
- Pub/Sub authenticated push subscriptions: https://cloud.google.com/pubsub/docs/authenticate-push-subscriptions

- Firestore transactions: https://firebase.google.com/docs/firestore/manage-data/transactions
- Gmail OAuth scopes: https://developers.google.com/workspace/gmail/api/auth/scopes
- Google OAuth 2.0 behavior: https://developers.google.com/identity/protocols/oauth2
- Google OAuth app states and production readiness: https://developers.google.com/identity/protocols/oauth2/production-readiness/overview
- OAuth verification exceptions for personal-use apps: https://support.google.com/cloud/answer/13464323